G.H. Raisoni College of Engineering and Management
Department of Mechanical Engineering

Unit 3 :Searching & Sorting Techniques

Prof. Sagar Mahajan

PART A : SEARCHING TECHNIQUES

Find
Locate
Get the Index !

1 What is Searching? (Concept)

Definition (Simplest Form):

Searching means finding the **position (index)** of a given element (called the **key**) in a collection of data.

If the element is present, we return its index; if not, we return **−1** (or "not found").

Key Terms:

- **Key** → the element we are searching for
- **Index** → position of the element
- **Not Found** → −1 (or message)

> Mechanical Mapping — The Tool Crib Problem

A storekeeper in a machine shop has 500 tools in a crib.
A worker asks:

"Where is the 12 mm end mill?"



The storekeeper must search the crib. How fast he finds it depends on how the tools are arranged — **randomly (linear search)** or **sorted** by size/part number (**binary search**).

> Why Searching Matters in Mechanical Engineering?

- **Looking up a value in a steam table** (sorted by temperature/pressure) → **Binary Search**
- Finding a part number in an inventory list → Depends on whether the list is sorted
- Locating a defective component on an assembly line → **Linear Scan**
- Searching a CNC tool database for the right tool → Depends on data organization

Mechanical Examples in Action

1 Steam Table Lookup (Binary Search)

Steam Table (Water)

Temperature (°C)	Pressure (bar)
100	1.01
150	4.76
200	15.54
250	39.75
300	85.88
...	...

Sorted data → use **Binary Search**

2 Part Number in Inventory (Depends on Sorting)

Part No.	Description	Qty
A102	Bearing	12
A117	Gear	8
B203	Shaft	5
C305	Coupling	20
D411	Bracket	7
...	...	

If sorted → **Binary Search**

If random → **Linear Search**

3 Defective Component (Assembly Line)



Scan one by one → **Linear Search**

4 CNC Tool Database (Depends on Organization)

Tool No.	Tool Name	Diameter
T01	End Mill	6 mm
T02	End Mill	8 mm
T03	End Mill	10 mm
T04	End Mill	12 mm
T05	Drill	5 mm
...	...	

If sorted → **Binary Search**
If random → **Linear Search**

Searching... Right tool + Right place = Faster work !

Same Idea...
Different Data
Different Speed!

LINEAR SEARCH (Sequential Search)

Check Each Element One by One!

Simple
Reliable
Works on Any List!

1 What is Linear Search?

Definition: Start from the first element and compare each element with the key one by one until either the key is found or the end of the list is reached.
Works on **any** list — **sorted** or **unsorted**.

2 Algorithm (Pseudocode)

```
LinearSearch(A, n, key):
  for i = 0 to n-1:
     if A[i] == key:
        return i   // found at index i
  return -1        // not found
```

3 Step-by-Step Example

Array: [45, 12, 78, 33, 90, 21, 67] Search for: 33

Step	Index	Value	Compare with 33	Result
1	0	45	45 ≠ 33	Continue
2	1	12	12 ≠ 33	Continue
3	2	78	78 ≠ 33	Continue
4	3	33	33 = 33	Found at index 3

Search for 99 (not present):

Compare all 7 elements → 7 comparisons → Not found

4 Time & Space Complexity

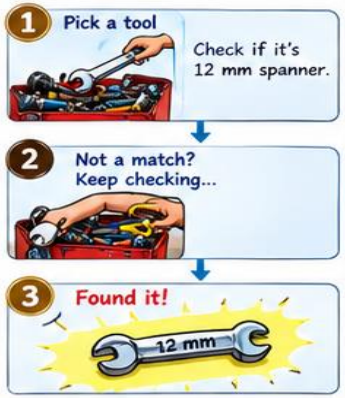
Case	Comparisons	Complexity
Best (key at index 0)	1	$O(1)$
Worst (key at last index or absent)	n	$O(n)$
Average	$n/2$	$O(n)$

Space Complexity

$O(1)$ — no extra memory needed.

5 Mechanical Mapping – Unsorted Toolbox





No shortcut is possible because the tools are not arranged in any order.

6 More Mechanical Examples



Checking each component on a conveyor for a defect (inspection station).



Finding a specific job card in an unsorted pile.



Searching for a part in an unsorted bin.

7 Advantages

- Works on unsorted data
- Simple to implement
- No preprocessing needed

8 Disadvantages

- Slow for large data — $O(n)$
- Does not exploit any order in the data

Linear Search...

Simple Idea, But Still Very Important!

Same Concept... Different Machines... Same Logic!

BINARY SEARCH (Divide & Conquer)

Find the element, by halving the search space!

Works Only on SORTED Array !

1 What is Binary Search?

Definition: Binary search works **only** on a **SORTED** array. It repeatedly divides the search interval in half.

1. Find the middle element.
2. If middle == key → found.
3. If key < middle → search left half.
4. If key > middle → search right half.
5. Repeat until found or interval is empty.

Important: mid = (low + high) / 2 uses integer division (floor). Show the calculation explicitly in exams.

2 Algorithm (Pseudocode)

```
BinarySearch(A, n, key):
    low = 0
    high = n - 1
    while low <= high:
        mid = (low + high) / 2   // integer division
        if A[mid] == key:
            return mid           // found
        else if A[mid] < key:
            low = mid + 1        // search right half
        else:
            high = mid - 1       // search left half
    return -1                    // not found
```

3 Step-by-Step Example (Search for 60)

Sorted Array:

10	20	30	40	50	60	70	80	90
0	1	2	3	4	5	6	7	8

Step	low	high	mid =(low+high)/2	A[mid]	Compare	Action
1	0	8	4	50	50 < 60	low = 5
2	5	8	6	70	70 > 60	high = 5
3	5	5	5	60	60 = 60	Found at index 5

Comparisons: 3 (vs 6 for linear search)

Search for 25 (not present)

10	20	30	40	50	60	70	80	90

Step	low	high	mid	A[mid]	Compare	Action
1	0	8	4	50	50 > 25	high = 3
2	0	3	1	20	20 < 25	low = 2
3	2	3	2	30	30 > 25	high = 1
4	2	1	—	—	low > high	Not found

Comparisons: 3 (then loop terminates)

4 Time & Space Complexity

Case	Comparisons	Complexity
Best (key at middle)	1	O(1)
Worst (key at end or absent)	$\lfloor \log_2 n \rfloor + 1$	O(log n)
Average	$\sim \log_2 n$	O(log n)

Space Complexity
O(1) iterative | O(log n) recursive

Why O(log n)?

Each comparison halves the search space.

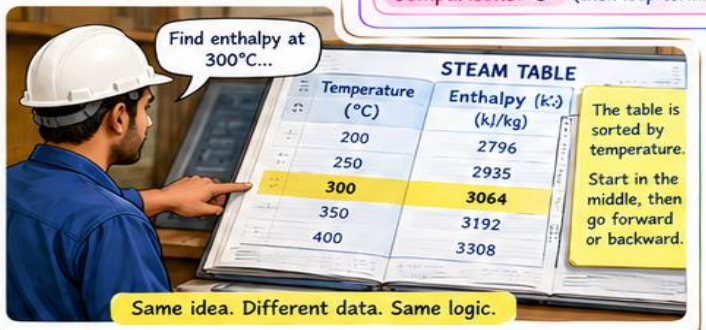
After k comparisons, remaining size = $n \frac{n}{2^k}$.

Stop when $\frac{n}{2^k} = 1$

$\rightarrow k = \log_2 n$.

For 1,000,000 elements ≈ 20 comparisons!

5 Mechanical Mapping – Sorted Steam Table



More Mechanical Examples

- Sorted bin of bolts (by diameter)
- Sorted inventory register
- Material properties list (e.g., densities)
- Calibration chart (sorted by pressure)

6 Advantages

- Very fast — O(log n)
- For 1,000,000 elements, only ~20 comparisons needed!

7 Disadvantages

- Requires sorted data (sorting costs O(n log n) one-time)
- Not suitable for linked lists (no random access)
- Data must be in contiguous memory (array)

8 Real-World Visual Summary

1. Look at middle

10	20	30	40	50	60	70	80	90

Middle

2. Go left or right

Key < 50 Search left half: 10 20 30 40

Key > 50 Search right half: 60 70 80 90

3. Repeat

Until found or empty.

Binary Search... Less Comparisons = Faster Results!

Sorted Data + Smart Search = Efficiency !

LINEAR SEARCH VS BINARY SEARCH COMPARISON

Choose the Right Tool !



Linear Search

Check each element one by one

Feature	Linear Search	Binary Search
Data requirement	Works on any data (sorted or unsorted)	Only sorted data
Approach	Check every element one by one	Divide search space in half
Best case	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(\log n)$
Average case	$O(n)$	$O(\log n)$
Data structure	Array or linked list	Array only (needs random access)
Preprocessing	None	Must sort first ($O(n \log n)$ one-time)
Mechanical analogy	Unsorted toolbox — check each tool	Sorted steam table — open near middle
When to use	Small data, unsorted, one-time search	Large data, sorted, many searches

Binary Search

Divide search space in half

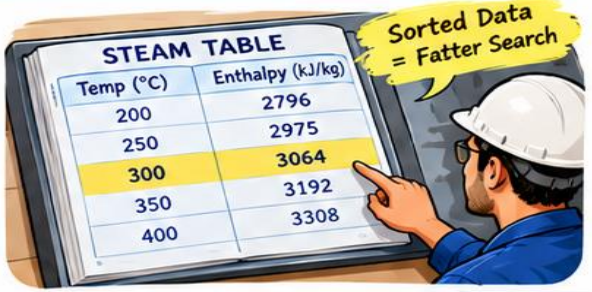




Rule of Thumb for Students

If you need to search **many times**, sort once and use **binary search**.

If you search **only once** on unsorted data, **linear search** is fine (sorting would cost more than it saves).



Same Goal → Different Paths → Choose Wisely !

Efficient Search = Better Decisions = Better Engineering !

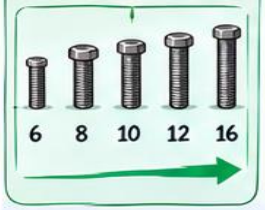
PART B: SORTING TECHNIQUES

Arrange Sort Organize In Order !

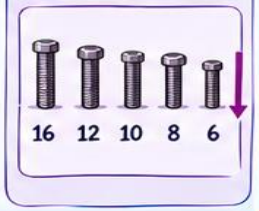
1 What is Sorting? (Concept)

Definition: Sorting means arranging a collection of elements in a specific order — usually ascending (smallest to largest) or descending (largest to smallest).

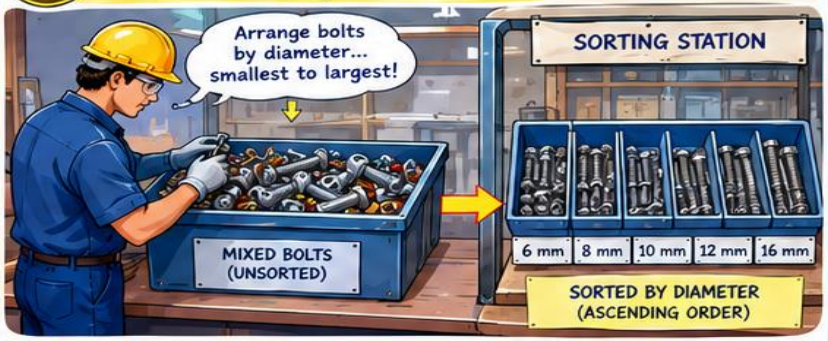
Ascending Order (Smallest → Largest)



Descending Order (Largest → Smallest)



2 Mechanical Mapping – The Bolt Sorting Station



Sorting is the process of arranging the bolts in order.

Different algorithms are different **strategies** for doing this — some compare adjacent bolts (bubble), some pick the smallest first (selection), some insert each bolt in its correct place (insertion), etc.

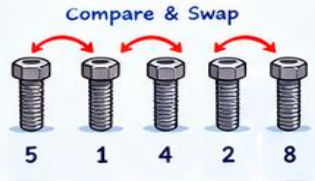
3 Why Sorting Matters

- Enables **binary search** (O(log n) instead of O(n))
- Makes data human-readable (sorted reports, inventory lists)
- Required for merge-based operations (merging two sorted lists)
- Used in scheduling (sort jobs by priority/deadline)
- Used in quality control (sort components by dimension for inspection)

Popular Sorting Algorithms (Quick Look)

1 Bubble Sort

Compare adjacent elements and swap if needed.



Simple, but slow ($O(n^2)$)

2 Selection Sort

Find the smallest element and place it at the beginning.



Fewer swaps ($O(n^2)$)

3 Insertion Sort

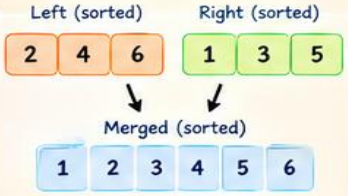
Pick each element and insert it into its correct place.



Good for small / nearly sorted data ($O(n^2)$)

4 Merge Sort

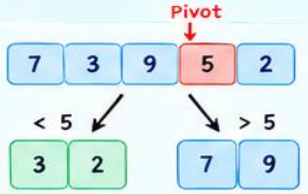
Divide, conquer, merge (sorted halves).



Fast & stable ($O(n \log n)$)

5 Quick Sort

Pick a pivot, partition, recursively sort.



Very fast (on average) ($O(n \log n)$)

Real-World Takeaway

Sorted data + Binary Search = Faster Results !

Same Data... Different Order... Big Difference!

BUBBLE SORT (Compare & Swap Adjacent Elements)

Largest element bubbles up to the end!

Simple Stable In-Place!

1 What is Bubble Sort? (Concept)

Definition: Repeatedly compare adjacent elements and swap them if they are in the wrong order. After each pass, the largest element "bubbles up" to the end of the array (like a large bubble rising to the top of a liquid).

Algorithm (Pseudocode)

```
BubbleSort(A, n):
  for i = 0 to n-2:          // n-1 passes
    for j = 0 to n-2-i:      // compare adjacent pairs
      if A[j] > A[j+1]:
        swap(A[j], A[j+1])
```

Optimized Version (Early Termination)

```
BubbleSort(A, n):
  for i = 0 to n-2:
    swapped = false
    for j = 0 to n-2-i:
      if A[j] > A[j+1]:
        swap(A[j], A[j+1])
        swapped = true
    if swapped == false:
      break      // already sorted — stop early
```

Stability: Yes (use >, not >=).

2 Step-by-Step Example

Array: [64, 34, 25, 12, 22, 11, 90]

Pass 1 – Largest (90) goes to the end

Comparison	Array after	Swap?
(64, 34)	[34, 64, 25, 12, 22, 11, 90]	Yes
(64, 25)	[34, 25, 64, 12, 22, 11, 90]	Yes
(64, 12)	[34, 25, 12, 64, 22, 11, 90]	Yes
(64, 22)	[34, 25, 12, 22, 64, 11, 90]	Yes
(64, 11)	[34, 25, 12, 22, 11, 64, 90]	Yes
(64, 90)	[34, 25, 12, 22, 11, 64, 90]	No

After Pass 1: [34, 25, 12, 22, 11, 64, 90] (90 at end)

Pass 2 – 2nd largest (64) in place

Comparison	Array after	Swap?
(34, 25)	[25, 34, 12, 22, 11, 64, 90]	Yes
(34, 12)	[25, 12, 34, 22, 11, 64, 90]	Yes
(34, 22)	[25, 12, 22, 34, 11, 64, 90]	Yes
(34, 11)	[25, 12, 22, 11, 34, 64, 90]	Yes
(34, 64)	[25, 12, 22, 11, 34, 64, 90]	No
(64, 90)	[25, 12, 22, 11, 34, 64, 90]	No

After Pass 2: [25, 12, 22, 11, 34, 64, 90] (64 in place)

Pass 3 – 3nd largest (34) in place

Comparison	Array after	Swap?
(25, 12)	[12, 25, 22, 11, 34, 64, 90]	Yes
(25, 22)	[12, 22, 25, 11, 34, 64, 90]	Yes
(25, 11)	[12, 22, 11, 25, 34, 64, 90]	Yes
(25, 34)	[12, 22, 11, 25, 34, 64, 90]	No
(34, 64)	[12, 22, 11, 25, 34, 64, 90]	No
(64, 90)	[12, 22, 11, 25, 34, 64, 90]	No

After Pass 3: [12, 22, 11, 25, 34, 64, 90] (34 in place)

Pass 4

Comparison	Array after	Swap?
(12, 22)	[12, 22, 11, 25, 34, 64, 90]	No
(22, 11)	[12, 11, 22, 25, 34, 64, 90]	Yes
(22, 25)	[12, 11, 22, 25, 34, 64, 90]	No
(25, 34)	[12, 11, 22, 25, 34, 64, 90]	No
(34, 64)	[12, 11, 22, 25, 34, 64, 90]	No
(64, 90)	[12, 11, 22, 25, 34, 64, 90]	No

After Pass 3: [12, 12, 25, 34, 64, 90]

Pass 5

Comparison	Array after	Swap?
(11, 11)	[11, 12, 22, 25, 34, 64, 90]	Yes
(22, 25)	[11, 12, 22, 25, 34, 64, 90]	No
(25, 34)	[11, 12, 22, 25, 34, 64, 90]	No
(34, 64)	[11, 12, 22, 25, 34, 64, 90]	No
(64, 90)	[11, 12, 22, 25, 34, 64, 90]	No

After Pass 5: [11, 12, 22, 25, 34, 64, 90] (Sorted)

Pass 6 – Early Termination — No swaps → Already sorted! ✓

Total Passes: 5 (or 6 with final check)

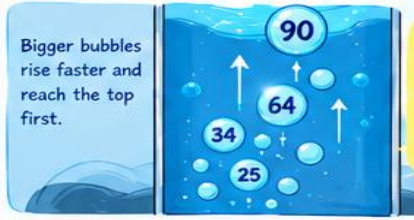
4 Time Complexity

Case	Comparisons	Swaps	Complexity
Best (already sorted)	n−1	0	O(n) (with optimization)
Worst (reverse sorted)	n(n−1)/2	n(n−1)/2	$O(n^2)$
Average	~n^2/2	~n^2/4	$O(n^2)$

Space Complexity: O(1) (in-place)

5 Mechanical Mapping – Bubbles Rising in a Fluid

Bigger bubbles rise faster and reach the top first.



Just like in bubble sort, the largest element "bubbles up" to the end in each pass.

More Mechanical Examples

- Vibrating sorting table (larger parts migrate to one end)
- Compare adjacent components on a conveyor (swap if out of order)
- Air bubbles in hydraulic fluid (larger rise to top)

6 Advantages

- Simple to understand and implement
- Stable sort
- In-place (O(1) space)
- Detects already-sorted data in O(n) (with optimization)

7 Disadvantages

- $O(n^2)$ — very slow for large data
- Many swaps (worst case n(n−1)/2)

Rule of Thumb

For large data → use better algorithms (Selection, Insertion, Merge, Quick, etc.).

Bubble sort is mainly for learning and small/simple cases.



Sorted Data = Faster Search = Better Results !

Same Data... Different Order... Big Difference!

SELECTION SORT (Select the Smallest & Place it!)

Find the minimum, swap, and grow the sorted part!

Simple Idea, Big Impact!

1 What is Selection Sort? (Concept)

Definition: Repeatedly select the smallest (or largest) element from the unsorted portion and place it at the correct position in the sorted portion. After each pass, the sorted portion grows by one element.

2 Algorithm (Pseudocode)

```
SelectionSort(A, n):
  for i = 0 to n-2:
      minIndex = i
      for j = i+1 to n-1:
          if A[j] < A[minIndex]:
              minIndex = j

      if minIndex != i:
          swap(A[i], A[minIndex])
```

Always selects the minimum from the remaining elements.

3 Step-by-Step Example

Array: [64, 25, 12, 22, 11]

Pass 1 (Find min in indices 0–4)

64	25	12	22	11
0	1	2	3	4

min = 11 (index 4)

Swap A[0] and A[4] →

11	25	12	22	64
0	1	2	3	4

Sorted portion: [11]

Pass 2 (Find min in indices 1–4)

11	25	12	22	64
0	1	2	3	4

min = 12 (index 2)

Swap A[1] and A[2] →

11	12	25	22	64
0	1	2	3	4

Sorted portion: [11, 12]

Pass 3 (Find min in indices 2–4)

11	12	25	22	64
0	1	2	3	4

min = 22 (index 3)

Swap A[2] and A[3] →

11	12	22	25	64
0	1	2	3	4

Sorted portion: [11, 12, 22]

Pass 4 (Find min in indices 3–4)

11	12	22	25	64
0	1	2	3	4

min = 25 (index 3) (already in place)

No swap needed →

11	12	22	25	64
0	1	2	3	4

Sorted portion: [11, 12, 22, 25]

Pass 5 (Only one element left)

11	12	22	25	64
0	1	2	3	4

Already sorted (no swap)

Final Array: [11, 12, 22, 25, 64]

Sorted!

4 Time Complexity

Case	Comparisons	Swaps	Complexity
Best (already sorted)	$n(n-1)/2$	0	$O(n^2)$
Worst (reverse sorted)	$n(n-1)/2$	$n-1$	$O(n^2)$
Average	$n(n-1)/2$	$\sim n/2$	$O(n^2)$

Key insight: Always does $n(n-1)/2$ comparisons regardless of input order.

5 Space Complexity

$O(1)$ — in-place

6 Stability

NOT stable

(Equal elements can change relative order during swaps.)

Example:

[4a, 4b, 2] → [2, 4b, 4a]

(4a and 4b swapped order)

7 Mechanical Mapping

Pickingathe Smallest Bolt



1. Scan the whole pile and pick the smallest bolt.



2. Place it in position 1.



3. Repeat for next smallest, and so on...



Unlike bubble sort (adjacent swaps), selection sort scans the whole remaining pile each time.

More Mechcal Examples

Quality inspection
Find the lightest component, place it in bin 1; then next lightest, etc.

Tool crib organization
Pick the smallest tool from the unsorted pile and place it in the sorted rack.

Grading apples by size
Pick the smallest apple, place it first; then next smallest, etc.

2 Advantages

- Simple to understand and implement
- Minimum number of swaps (at most $n-1$) — good when swap cost is high (e.g., large records)
- In-place ($O(1)$ space)
- Predictable performance (always $O(n^2)$ comparisons)

8 Disadvantages

- $O(n^2)$ — slow for large data
- Not stable
- Cannot detect already-sorted data (always does full comparisons)

Selection Sort — Simple • Predictable • But Slow!

Good for small data or when swaps are costly!

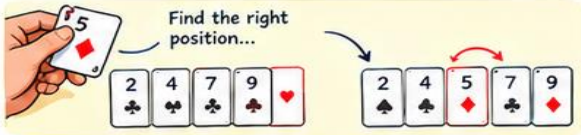
INSERTION SORT (Build Sorted Array One Element at a Time)

Just like inserting a card into a sorted hand !

Simple Idea... Powerful Results!

1 What is Insertion Sort? (Concept)

Definition: Build the sorted array one element at a time. Take each element from the unsorted portion and insert it into its correct position in the already-sorted portion — just like inserting a card into a sorted hand of playing cards.



Key idea: Take one element, find its place, insert it, and move to the next element.

2 Algorithm (Pseudocode)

```
InsertionSort(A, n):
    for i = 1 to n-1:
        key = A[i]
        j = i - 1
        while j >= 0 and A[j] > key:
            A[j+1] = A[j]        // shift right
            j = j - 1
        A[j+1] = key             // insert key
```

3 Step-by-Step Example

Array: [11, 13, 5, 6]

Step 1 – Insert 11

Key = 11 (index 1)

12	11	13	5	6
0	1	2	3	4

1. 11 < 12 → shift 12 right
2. Insert 11 at index 0

Array: [11, 12, 13, 5, 6]

Sorted: [11, 12] ✓

Step 2 – Insert 13

Key = 13 (index 2)

11	12	13	5	6
0	1	2	3	4

13 > 12 → stays in place

Array: [11, 12, 13, 5, 6]

Sorted: [11, 12, 13] ✓

Step 3 - Insert 5

Key = 5 (index 3)

11	12	13	5	6
0	1	2	3	4

1. 5 < 13 → shift 13
2. 5 < 12 → shift 12
3. 5 < 11 → shift 11
4. Insert 5 at index 0

Array: [5, 11, 12, 13, 6]

Sorted: [5, 11, 12, 13] ✓

Step 4 – Insert 6

Key = 6 (index 4)

5	11	12	13	6
0	1	2	3	4

1. 6 < 13 → shift 13
2. 6 < 12 → shift 12
3. 6 < 11 → shift 11
4. 6 > 5 → stop
5. Insert 6 at index 1

Array: [5, 6, 11, 12, 13]

Sorted: [5, 6, 11, 12, 13] ✓

Step 5 - Done

Only one element left (index 4)

5	6	11	12	13
0	1	2	3	4

Sorted!

Final Array: [5, 6, 11, 12, 13]

4 Time Complexity

Case	Comparisons	Shifts	Complexity
Best (already sorted)	n – 1	0	$O(n)$
Worst (reverse sorted)	n(n – 1)/2	n(n – 1)/2	$O(n^2)$
Average	~$n^2/4$	~$n^2/4$	$O(n^2)$

Key insight:
Best case is O(n) (already sorted).
Worst and average cases are O(n^2).

5 Space Complexity

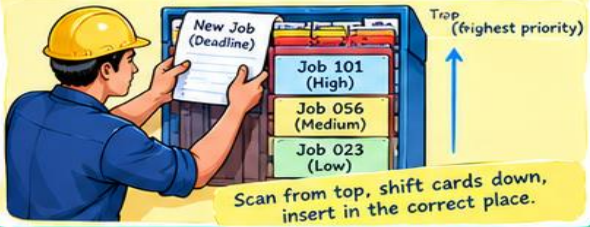
O(1) – in-place

6 Stability

Stable – equal elements keep their relative order (we only shift when A[j] > key).

6 Mechanical Mapping

Inserting Job Cards in a Sorted Stack



Scan from top, shift cards down, insert in the correct place.

7 More Mechanical Examples



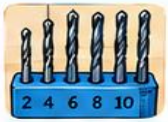
Arrange tools on a pegboard (in order)



Insert a new part into a sorted rack



Sort job cards by deadline



Organize drill bits by size

8 Advantages

- Simple and intuitive
- Stable sort
- In-place (O(1) space)
- Efficient for small / nearly-sorted data (O(n) best case)
- Online algorithm (works as data arrives)
- Used in hybrid sorts (e.g., Timsort)

9 Disadvantages

- $O(n^2)$ for large data
- Many shifts (worst case n(n–1)/2)

Small data... Sorted fast!

Insertion Sort → Simple Steps → Correct Position → Sorted Array !

MERGE SORT (Divide and Conquer)

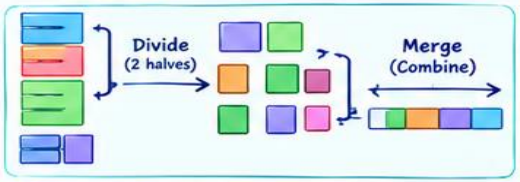
Divide → Solve → Merge

Small Problems + Smart Merging = Big Results!

1 What is Merge Sort? (Concept)

Definition: Merge sort uses the divide-and-conquer strategy:

1. **Divide** the array into two halves.
2. **Recursively sort** each half.
3. **Merge** the two sorted halves into one sorted array.



Key idea: Break the problem into smaller parts, solve them, and merge the results.

2 Algorithm (Pseudocode)

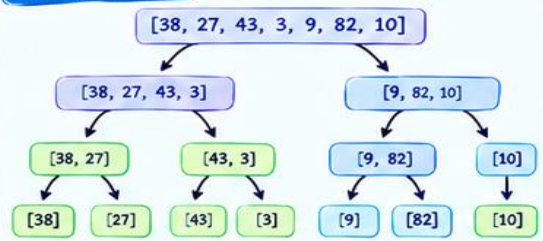
```
MergeSort(A, low, high):
  if low < high:
      mid = (low + high) / 2
      MergeSort(A, low, mid)      // sort left half
      MergeSort(A, mid + 1, high) // sort right half
      Merge(A, low, mid, high)    // merge both halves
```

```
Merge(A, low, mid, high):
  Create temporary arrays L = A[low..mid], R = A[mid+1..high]
  i = 0, j = 0, k = low
  while i < len(L) and j < len(R):
      if L[i] <= R[j]:
          A[k++] = L[i++]
      else:
          A[k++] = R[j++]
  Copy remaining elements of L and R into A
```

3 Step-by-Step Example

Array: [38, 27, 43, 3, 9, 82, 10]

Divide Phase



Merge Phase (Bottom-up)

1. [38] + [27] → [27, 38] [43] + [3] → [3, 43]
2. [27, 38] + [3, 43] → [3, 27, 38, 43]
3. [9] + [82] → [9, 82] [9, 82] + [10] → [9, 10, 82]
4. [3, 27, 38, 43] + [9, 10, 82] → [3, 9, 10, 27, 38, 43, 82]

Final Sorted Array

[3, 9, 10, 27, 38, 43, 82] ✓

Merging Two Sorted Arrays – Detailed Example

Merge [3, 27, 38, 43] and [9, 10, 82]

Step	i (left)	j (right)	Compare	Pick	Result
1	3	9	3 < 9	3	[3]
2	27	9	27 > 9	9	[3, 9]
3	27	10	27 > 10	10	[3, 9, 10]
4	27	82	27 < 82	27	[3, 9, 10, 27]
5	38	82	38 < 82	38	[3, 9, 10, 27, 38]
6	43	82	43 < 82	43	[3, 9, 10, 27, 38, 43]
7	—	82	Left exhausted	82	[3, 9, 10, 27, 38, 43, 82]

4 Merge Sort – Time & Space

Case	Complexity
Best	$O(n \log n)$
Worst	$O(n \log n)$
Average	$O(n \log n)$

Why $O(n \log n)$?

- Divide $\log_2 n$ times (halving each time).
- Each level takes $O(n)$ to merge.
- Total = $O(n \log n)$.

Space Complexity

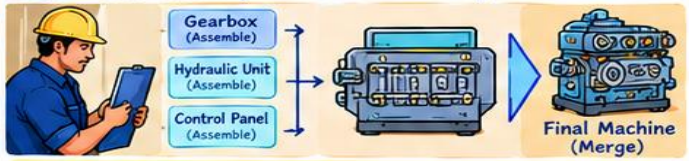
$O(n)$ – requires temporary arrays (for merging).

Stability

Stable – pick from left first (if equal).

5 Mechanical Mapping

Sub-Assembly Then Final Assembly



Divide the large machine into sub-assemblies, assemble each, then merge into the final machine.

6 More Mechanical Examples

- Two sorted conveyor lines merging into one.
- Dividing a large batch of parts for parallel processing, then combining.
- Merge two sorted inventory lists into one master list.

7 Advantages

- Guaranteed $O(n \log n)$ performance.
- Stable sort.
- Works well for linked lists (no random access needed).
- Predictable performance.
- Good for external sorting (large data).

8 Disadvantages

- $O(n)$ extra space (not in-place).
- Recursive overhead.
- Slower than quick sort for small arrays.

Merge Sort → Divide → Sort → Merge → Sorted Array !

Efficient. Stable. Perfect for Big Data!

QUICK SORT (Divide and Conquer)

Pick a Pivot → Partition → Recursively Sort

Fast in Practice! Smart Pivot = Big Difference!

1 What is Quick Sort? (Concept)

Definition: Quick sort also uses divide-and-conquer:

1. Choose a **pivot** element.
2. **Partition the array**: elements smaller than pivot go left, larger go right.
3. Recursively sort the left and right sub-arrays.



Key idea: Partition around a pivot, then solve smaller problems recursively.

2 Algorithm (Pseudocode)

```
QuickSort(A, low, high):
  if low < high:
      pivotIndex = Partition(A, low, high)
      QuickSort(A, low, pivotIndex - 1)
      QuickSort(A, pivotIndex + 1, high)

Partition(A, low, high):
  pivot = A[high]        // choose last element as pivot
  i = low - 1
  for j = low to high - 1:
     if A[j] <= pivot:
          i = i + 1
          swap(A[i], A[j])
  swap(A[i+1], A[high])  // place pivot in correct position
  return i + 1
```

3 Step-by-Step Example

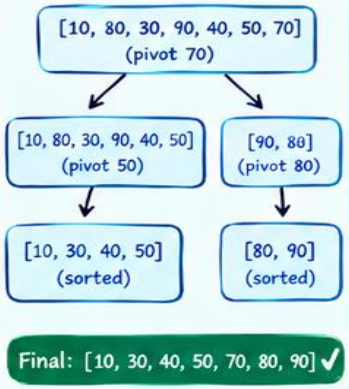
Array: [10, 80, 30, 90, 40, 50, 70]

Pivot = last element (70)

j	A[j]	A[j] ≤ 70?	Action	Array after	i
0	10	Yes	i=0, swap(0,0)	[10, 80, 30, 90, 40, 50, 70]	0
1	80	No	-	[10, 80, 30, 90, 40, 50, 70]	0
2	30	Yes	i=1, swap(1,2)	[10, 30, 80, 90, 40, 50, 70]	1
3	90	No	-	[10, 30, 80, 90, 40, 50, 70]	1
4	40	Yes	i=2, swap(2,4)	[10, 30, 40, 90, 80, 50, 70]	2
5	50	Yes	i=3, swap(3,5)	[10, 30, 40, 50, 80, 90, 70]	3
End	-	-	swap(4,6)	[10, 30, 40, 50, 70, 90, 80]	-

Pivot 70 is now at index 4 — left ≤ 70, right > 70.

4 Recursion in Action



Final: [10, 30, 40, 50, 70, 80, 90] ✓

5 Partition Visualization (Lomuto Partition)

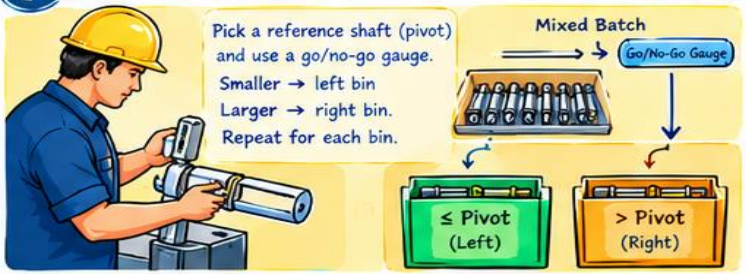


6 Merging Two Sorted Arrays (Detailed Example)

L = [3, 27, 38, 43] R = [9, 10, 82]

Step	i (left)	j (right)	Compare	Pick	Result
1	0	0	3 < 9	3	[3]
2	1	0	27 > 9	9	[3, 9]
3	1	1	27 > 10	10	[3, 9, 10]
4	1	2	27 < 82	27	[3, 9, 10, 27]
5	2	2	38 < 82	38	[3, 9, 10, 27, 38]
6	3	2	43 < 82	43	[3, 9, 10, 27, 38, 43]
7	–	2	Left exhausted	82	[3, 9, 10, 27, 38, 43, 82]

7 Mechanical Mapping – Go/No-Go Gauge Partitioning



Pick a reference shaft (pivot) and use a go/no-go gauge.
Smaller → left bin
Larger → right bin.
Repeat for each bin.

Like sorting parts by a reference dimension, or dividing a batch by a threshold value.

8 More Mechanical Examples

- Sorting parts by a reference dimension (smaller / larger).
- Triage on a shop floor (critical / routine jobs).
- Dividing a batch by a threshold (e.g., above/below tolerance).

9 Advantages &

- Fast in practice (good cache performance).
- In-place (O(log n) stack space).
- Average O(n log n).
- Widely used in standard libraries (e.g., C's qsort).
- Good with large datasets.

9 Disadvantages

- Worst case $O(n^2)$ (poor pivot choice).
- Not stable.
- Recursive (stack overhead).
- Performance depends on pivot selection.

Use **median-of-three** or random pivot to avoid **worst case.**

Quick Sort → Divide → Partition → Recursively Sort → Sorted Array !

Same Idea... Different Order... Big Impact!

COMPARISON OF ALL SORTING ALGORITHMS

Same Goal → Different Strategies → Different Strengths

Choose the right tool for the right job!

Algorithm	Best (Complexity)	Average (Complexity)	Worst (Complexity)	Space (Complexity)	Stable?	In-place?	Mechanical Hook
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	 Bubbles rising in fluid
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes	 Pick smallest bolt each time
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	 Insert job card in sorted stack
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	 Sub-assembly then final assembly
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes	 Go/no-go gauge partitioning

When to Use Which? (Placement / Exam Tip)

Situation	Best Choice	Why?
Small data ($n < 20$)	Insertion sort	Low overhead, fast for small n
Nearly sorted data	Insertion sort	$O(n)$ best case
Need stable sort	Merge sort / Insertion / Bubble	Preserves equal-element order
Memory constrained	Quick sort / Insertion / Selection	In-place
Guaranteed performance	Merge sort	Always $O(n \log n)$
General-purpose, fast	Quick sort	Fastest average in practice
Linked list sorting	Merge sort	No random access needed
External sorting (huge data)	Merge sort	Sort chunks, then merge

Key Takeaways

- Bubble, Insertion, Merge = Stable
- Selection, Quick = Not stable
- In-place: Bubble, Selection, Insertion, Quick
- Merge needs $O(n)$ extra space
- Quick sort is fastest on average
- Merge sort gives guaranteed $O(n \log n)$
- Insertion sort shines on small / nearly sorted data.

Remember

It's not just about the algorithm...

It's about the right algorithm for the right situation!



Sort Smarter → Not Harder !

Right Algorithm... Better Results !